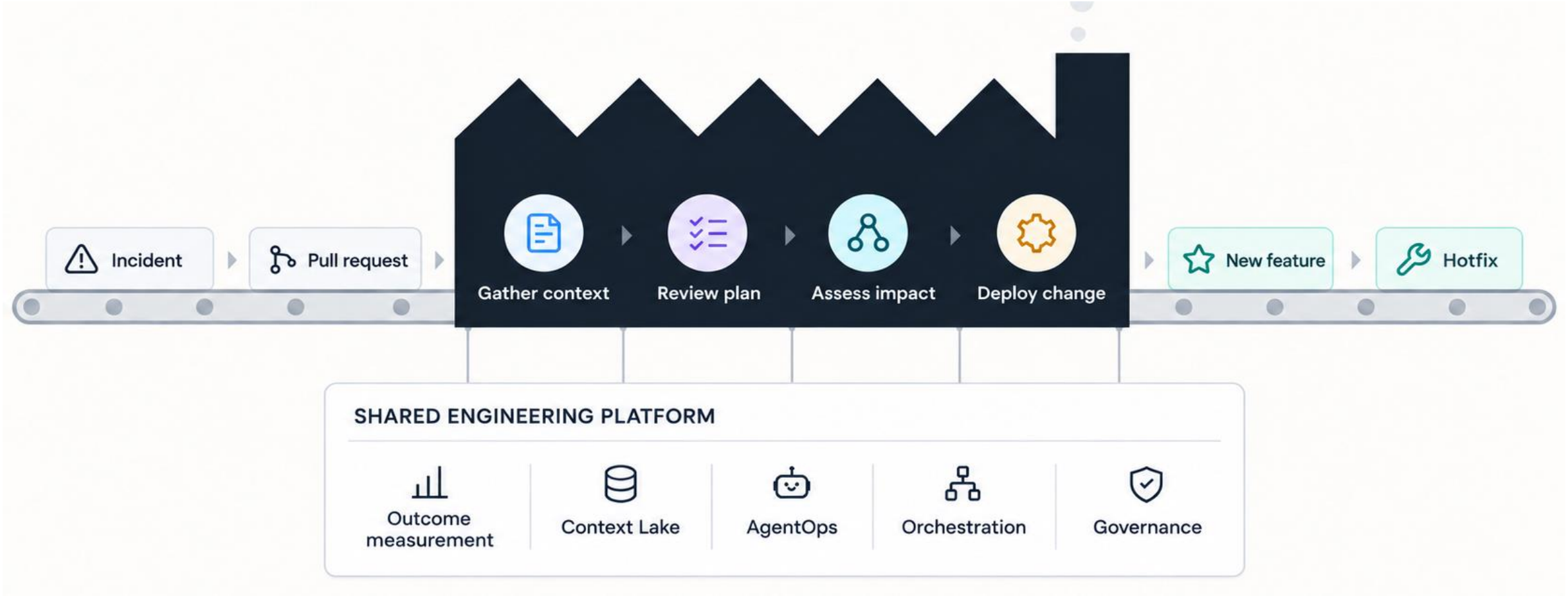
From IDP to Software Factory

Build, govern, and operate agentic development



German Martinez

The Journey

01 The IDP foundation	Make software understandable and common work repeatable.
02 The agentic shift	Give agents the environment to carry engineering tasks.
03 The software factory	Connect factory lines across the software lifecycle.
04 Build, govern, operate	Provide shared capabilities around agents and automation.
05 Grow from your IDP	Extend existing foundations and expand with evidence.

A platform built for developers becomes a foundation for developers and agents.

The Developer Coordination Problem

Information is scattered

Ownership, architecture, deployments, and operational data live in different systems.

Routine work crosses team boundaries

A new environment or service can require tickets, permissions, and manual handoffs.

Standards are difficult to apply consistently

Teams must discover the right patterns and keep up as requirements change.

An IDP gives teams a supported way to understand their software and get work done.

What Is an Internal Developer Platform?

An internal product that brings shared engineering capabilities together so teams can build, deploy, and operate software through supported paths.

The platform

Shared services, automation, APIs, policies, and operational context.

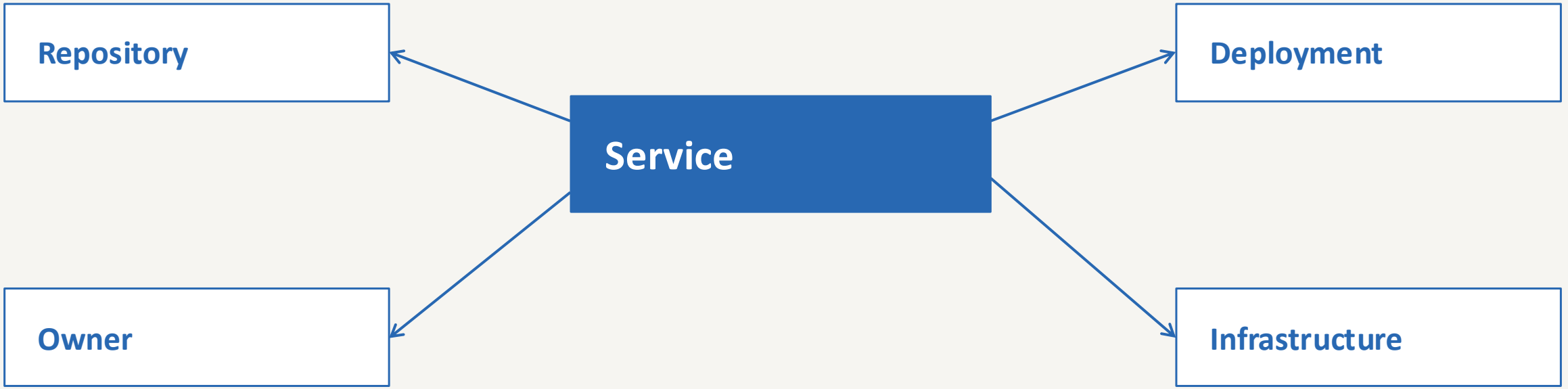
The portal

A developer-facing interface for discovery and self-service.

The platform team

Owns the experience, maintains capabilities, and improves them with users.

The Software Catalog Makes Context Useful



Relationships connect software to the people and resources behind it.

Answer: What exists? Who owns it? Where does it run? What depends on it?

Self-Service Turns Context Into Action

A developer requests an outcome

Create a service, provision an environment, or run a supported operational action.

The platform coordinates existing automation

Collect inputs, validate access, and invoke CI/CD, infrastructure as code, or APIs.

The result stays connected to the service

Return status and outputs, record ownership, and link newly created resources.

Golden paths provide maintained patterns, useful defaults, and clear choices.

Autonomy With Organizational Standards

Golden paths make approved practices reusable

Templates and workflows encode supported configurations and operational defaults.

Policies and permissions define boundaries

Control who can do what and where approval is required.

Scorecards make expectations visible

Show service readiness, missing requirements, and actions that close the gaps.

Benefits: fewer handoffs, less repeated setup, and more consistent engineering practices.

Agents Join the Engineering Team

As agents take on complete tasks, they need context, tools, and a supported way to work.

From AI Assistance to Delegated Work

Assist

Explain code. Suggest a patch. Generate a test.

The developer coordinates the work.

Delegate

Investigate an issue. Implement a change. Run checks. Open a PR.

The agent owns a defined task.

Coordinate

Connect tasks across agents, automation, and human decisions.

The workflow manages progression.

The architecture must support tasks that span several tools and stages.

Give Agents an Engineering Environment

A clear assignment	A mission, acceptance criteria, and stopping conditions.
System knowledge	Relevant code, service context, ownership, and runtime data.
Tools and authority	Development environments and scoped, permissioned actions.
A way to prove the work	Tests, reviews, deployment checks, and outcome verification.

Owning an end-to-end task can still include review and approval boundaries.

Agentic Chaos

Developers build in silos

Teams duplicate agents, skills, and integrations. Knowledge stays local.

No shared standards

Ownership, evaluation, and maintenance practices vary between teams.

Messy context

Stale, fragmented, or conflicting data makes reliable decisions harder.

No consistent guardrails

Access, approvals, and execution limits depend on each implementation.

The result: duplicated effort, unpredictable quality, and limited trust in delegated work.

Engineering Work Becomes Factory Lines

Connect people, agents, and automation across the software lifecycle.

What Is an AI Software Factory?

An integrated engineering system where people, agents, and automation carry work across the software lifecycle through repeatable workflows.

Shared context makes work relevant.
Governance defines how it may proceed.
Operational feedback shows whether it succeeded.

The factory combines engineering execution with the environment that supports it.

Factory Lines Across the Software Lifecycle

01

Idea to PR to production

Plan, implement, test, and progress a change.

02

Agentic infrastructure management

Propose, provision, and verify infrastructure.

03

Self-healing incidents

Investigate, remediate, and verify recovery.

04

Agentic end of life

Assess dependencies and retire resources.

05

Auto-fixed CI/CD

Diagnose failures, repair, and rerun checks.

06

Auto-patching CVEs

Prioritize findings and deliver verified fixes.

Each line combines reasoning, automation, human decisions, and verification.

A Shared Platform to Build, Govern, and Operate

BUILD

Connect context, agents, tools, and automation into reusable factory lines.

GOVERN

Define ownership, access, standards, and human decision points.

OPERATE

Observe execution, verify outcomes, and improve quality and cost.

A unified platform can coordinate multiple agent tools and execution systems.

Build. Govern. Operate.

Shared capabilities turn isolated agent workflows into an engineering system teams can maintain.

Build: Context and Orchestration

Supply relevant, permissioned context

Combine service relationships with current data from the systems that own it.

Compose agents with deterministic automation

Use reasoning for investigation and adaptation; use explicit logic for defined checks.

Maintain reusable skills and workflow patterns

Version instructions, tool contracts, and execution paths with clear owners.

Example: investigate a failed pipeline, propose a repair, then rerun existing CI checks.

Govern: Identity, Standards, and Decisions

Give every agent and workflow an owner

Maintain a registry of agents, skills, and tools with readiness criteria.

Enforce scoped access at execution time

Use identities, permission boundaries, and policy checks outside the prompt.

Route human decisions according to risk

Present the plan, change, checks, and remaining uncertainty to the right reviewer.

Example: a production infrastructure change requires designated approval before execution.

Operate: Evidence and Continuous Improvement

Trace requests through decisions and actions

Record execution history, relevant versions, tool calls, and outcomes.

Handle failures and evaluate changes

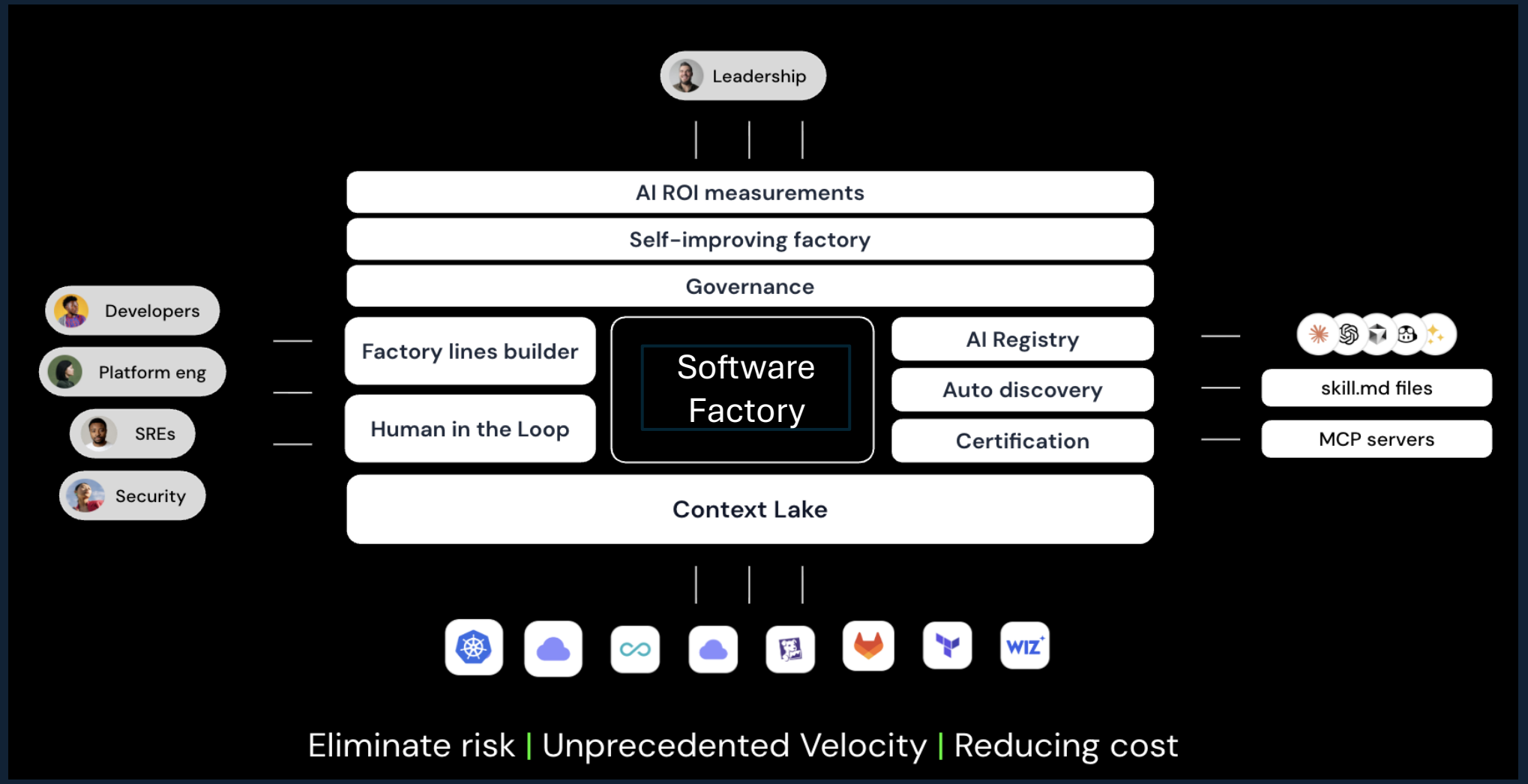
Bound retries, route escalation, and evaluate new models, prompts, and skills.

Measure useful results and their cost

Track accepted outcomes, elapsed time, human effort, rework, and execution cost.

Example: a merged patch is followed through deployment and remediation confirmation.

The Unified Software Factory Architecture



Your IDP Is the Starting Point

Extend existing context, self-service, and controls as agents take on more responsibility.

Your IDP Is the Foundation

Catalog + relationships	→	Relevant context for agent decisions
Self-service actions	→	Supported operations for agent execution
Golden paths	→	Reusable factory workflow patterns
Ownership + access	→	Accountability and authority boundaries
Scorecards + visibility	→	Readiness, outcome, and quality feedback

Extend the foundations you have, then add the capabilities your workflows need.

Grow Responsibility and Autonomy Gradually

1. Connect	Establish useful context, ownership, and supported actions.
2. Delegate	Give an agent one bounded task with explicit review.
3. Coordinate	Connect agents, automation, and human decisions into a line.
4. Expand	Broaden scope or autonomy when operating evidence supports it.

Choose a recurring task with clear acceptance criteria and a manageable recovery path.

Benefits You Can Measure

Less coordination	Manual handoffs and time waiting for routine requests.
Reusable capabilities	Repeated implementation work across teams.
Consistent controls	Policy coverage, ownership, and traceable execution.
Useful delegation	Accepted outcomes, rework, human effort, and cost.

Establish a baseline and compare equivalent work. Improvements are outcomes to test.

A Platform for Developers and Agents

The foundations stay valuable.
The scope of work expands.

IDP: help teams understand and change their software.

Software factory: enable people, agents, and automation to carry engineering work across the lifecycle.

Build. Govern. Operate.

Questions?